



Bahria University, Islamabad
Department of Software Engineering

Machine Learning Lab
(Fall-2026)

Teacher: Engr. Kamran Khan

Student Name: Faisal khan

Enrollment: 01-131232-105

Lab Journal: 03

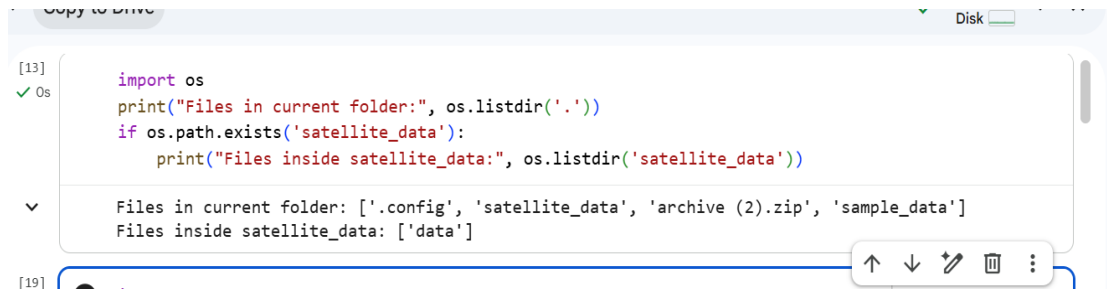
Date: 15-09-2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

Demonstrate various data pre-processing techniques for a given dataset



```
[13] ✓ Os
import os
print("Files in current folder:", os.listdir('.'))
if os.path.exists('satellite_data'):
    print("Files inside satellite_data:", os.listdir('satellite_data'))

Files in current folder: ['.config', 'satellite_data', 'archive (2).zip', 'sample_data']
Files inside satellite_data: ['data']
```

```
[19] .

import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical
import random

# Path to extracted satellite dataset
data_dir = './satellite_data/data'
IMG_SIZE = 128
categories = ['cloudy', 'desert', 'green_area', 'water']

data = []
labels = []

print("Loading and inspecting images...")
for category in categories:
    path = os.path.join(data_dir, category)
    class_num = categories.index(category)

    if not os.path.exists(path):
        continue

    for img_name in os.listdir(path):
        img_path = os.path.join(path, img_name)
        try:
            # Read image using OpenCV
            img_arr = cv2.imread(img_path)

            # Handle corrupted or missing images
            if img_arr is None:
                continue

            # Resize and convert color space (BGR to RGB)
            resized_img = cv2.resize(img_arr, (IMG_SIZE, IMG_SIZE))
            resized_img = cv2.cvtColor(resized_img, cv2.COLOR_BGR2RGB)

            data.append(resized_img)
```

```
        labels.append(class_num)
    except Exception as e:
        pass

X = np.array(data)
y = np.array(labels)

print(f"Total valid images loaded: {X.shape[0]}")

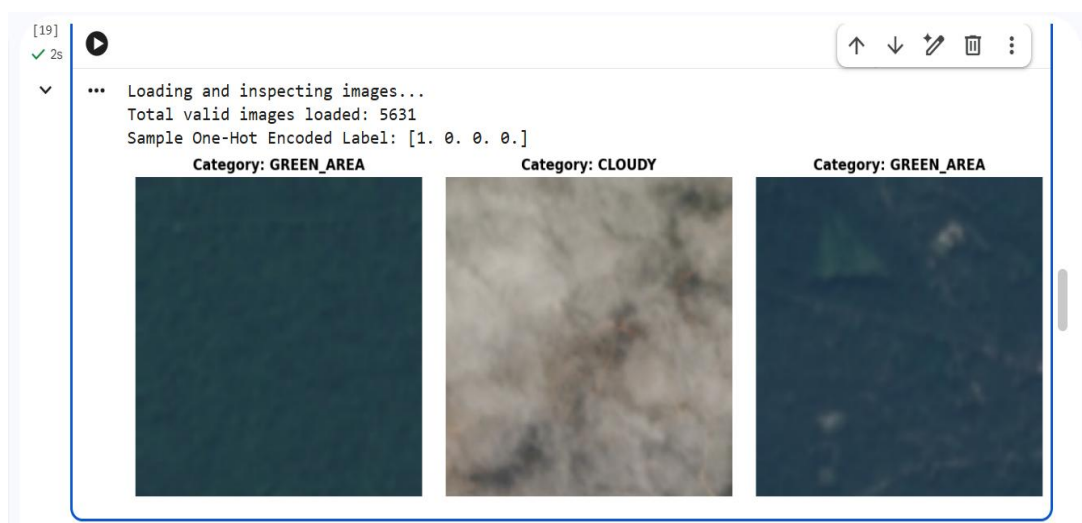
# One-Hot Encoding Categorical Labels
y_encoded = to_categorical(y, num_classes=len(categories))
print("Sample One-Hot Encoded Label:", y_encoded[0])

# Display 3 random images with their actual category names for your
instructor
if X.shape[0] >= 3:
    random_indices = random.sample(range(len(X)), 3)

    plt.figure(figsize=(12, 4))
    for i, index in enumerate(random_indices):
        plt.subplot(1, 3, i + 1)
        plt.imshow(X[index])

        cat_name = categories[np.argmax(y_encoded[index])]
        plt.title(f"Category: {cat_name.upper()}", fontsize=12,
fontweight='bold')
        plt.axis('off')

    plt.tight_layout()
    plt.show()
else:
    print("Not enough images loaded.")
```



```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# 1. Flatten the image data from 3D (samples, height, width, channels)
to 2D tabular format
num_samples = X.shape[0]
X_flattened = X.reshape(num_samples, -1).astype(np.float32)

# 2. Apply MinMaxScaler (Scales pixel values strictly between 0.0 and
1.0)
min_max_scaler = MinMaxScaler()
X_minmax = min_max_scaler.fit_transform(X_flattened)

# 3. Apply StandardScaler (Centers data around mean=0 with standard
deviation=1)
standard_scaler = StandardScaler()
X_standard = standard_scaler.fit_transform(X_flattened)

# Print verification statistics for your lab report
print("Feature scaling completed successfully!")
print(f"Original Pixel Range: min = {X_flattened.min()}, max =
{X_flattened.max()}")
print(f"MinMaxScaler Range: min = {X_minmax.min():.2f}, max =
{X_minmax.max():.2f}")
print(f"StandardScaler Stats -> Mean = {X_standard.mean():.2f}, Std =
{X_standard.std():.2f}")

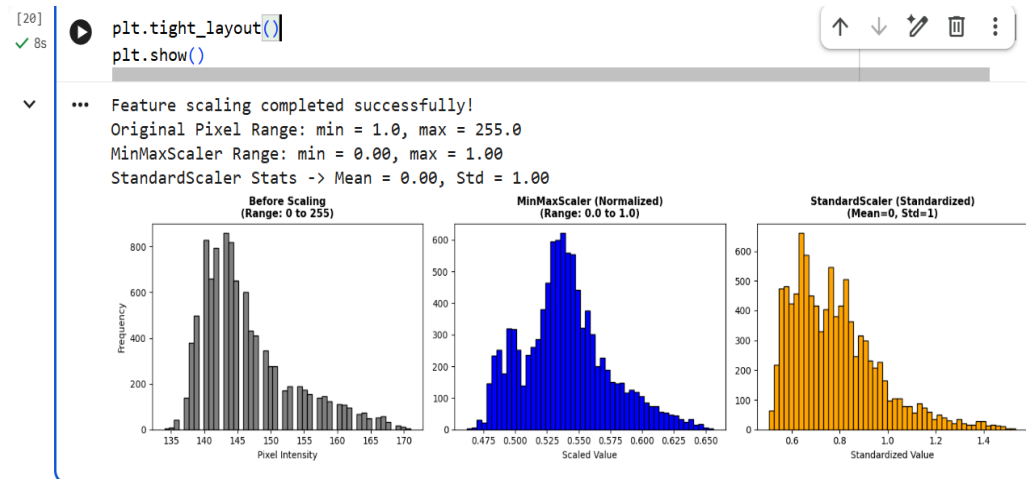
# 4. Visualization: Plot pixel distributions before and after scaling
plt.figure(figsize=(15, 4))

# Before Scaling Distribution
plt.subplot(1, 3, 1)
plt.hist(X_flattened.ravel()[:10000], bins=50, color='gray',
edgecolor='black')
plt.title("Before Scaling\n(Range: 0 to 255)", fontsize=11,
fontweight='bold')
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

# After MinMaxScaler Distribution
plt.subplot(1, 3, 2)
plt.hist(X_minmax.ravel()[:10000], bins=50, color='blue',
edgecolor='black')
plt.title("MinMaxScaler (Normalized)\n(Range: 0.0 to 1.0)", fontsize=11,
fontweight='bold')
plt.xlabel("Scaled Value")

# After StandardScaler Distribution
plt.subplot(1, 3, 3)
```

```
plt.hist(X_standard.ravel()[:10000], bins=50, color='orange',  
edgecolor='black')  
plt.title("StandardScaler (Standardized)\n(Mean=0, Std=1)", fontsize=11,  
fontweight='bold')  
plt.xlabel("Standardized Value")  
  
plt.tight_layout()  
plt.show()
```



Thank you
